

ApplyJin — AI Engineering Gap Analysis & Implementation Plan

ApplyJin — AI Engineering Gap Analysis & Implementation Plan

Status: audited against current main (post 7e41b88) **Verdict:** Do not rewrite ApplyJin. The architecture is now good enough to harden. The next phase should turn the existing components into one measurable, reproducible inference system.

Legend:  implemented ·  partial/buggy ·  not implemented ·  backlog item

0. What ApplyJin is becoming

The project is no longer Job → LLM → Resume. The current architecture is a staged inference system:

```
JOB → JD ANALYSIS → FIT SCORING → LAYA GATE → {SKIP | REVIEW |  
GENERATE}  
  → RETRIEVAL → CONTEXT BUILDER → LLM ROUTER → {vLLM | LiteLLM |  
fallback}  
  → GENERATION → CLAIM EXTRACTION → VERIFICATION → {BLOCK | PASS}  
  → TRACKER → EVALUATION
```

This is a legitimate AI-systems architecture. The remaining work is primarily about correctness, measurement, integration, and experimental rigor — not new frameworks.

1. What is already good

- Clean separation of **decision system** (DecisionRequest → DecisionProvider → Laya/heuristic → deterministic policy → GENERATE/REVIEW/SKIP) from the **action system**. The model never decides whether ApplyJin performs an application action; the policy layer does. 
- Typed decision outputs, fallback behavior, hashed decision traces, explicit policy versions. 
- Context boundaries with untrusted-content delimiters (<UNTRUSTED_JOB_DESCRIPTION>, <VERIFIED_CANDIDATE_EVIDENCE>). 

- Typed claim verification (metrics, dates, skills, companies, titles, certifications, education) with known_entities.  (commit 68a5182)
- Canonical provider unification: LLMRouter dispatches every chain entry through one LLMProvider per entry (LiteLLMProvider / VLLMProvider) behind a single background-loop runner.  (commit 7e41b88)
- Metrics: Prometheus endpoint, SQLite inference store, generation telemetry (TTFT/TPOT/tokens/failures), retrieval metrics (Recall@k/MRR/NDCG), decision metrics. 
- Evaluation harness: offline JSONL evaluation, reports, regression gate, baselines. 
- vLLM benchmark tooling, quantization comparison, QLoRA smoke experiment, held-out split, failure taxonomy, human-review fallback. 

Conclusion: make every important claim about ApplyJin measurable.

2. The main remaining problem: integration maturity

Good component + good component + good component $\neq$ good system. You must prove the entire chain works:

JOB $\rightarrow$ decision $\rightarrow$ retrieval $\rightarrow$ context $\rightarrow$ model $\rightarrow$ resume $\rightarrow$ verification

Does the complete system produce better, safer, faster output? That is the next research/engineering question.

3. Fact-check of the audit claims against current main

Claim	Verdict	Evidence
Routing mixes raw milliseconds with 0–1 quality	 stale	hermes/inference/router.py bounds latency_score = latency_ms/latency_budget_ms and resource_score = resource_cost/resource_budget into [0,1] before weighting
Context budgets are characters, text[:budget]	 partly true	estimate_tokens() and input_tokens/budget_tokens exist, but truncation is inconsistent: job uses text[:budget*4] (budget=tokens), evidence uses budget//4 then $\times 4$ (budget=chars), and budget_tokens = budget_characters//4 mislabels the budget sum. Unit accounting must be fixed
Router receives len(prompt) characters	 stale	hermes/agents/resume_tailor.py:207 passes context_length=context.input_tokens (estimated tokens)
Verification only GENERAL/METRIC/DATE	 stale	taxonomy now includes CERTIFICATION, EDUCATION, SKILL, COMPANY, TITLE

Claim	Verdict	Evidence
Two LiteLLM/vLLM abstractions	✗ stale	(commit 68a5182) consolidated behind one LLMPProvider per chain entry (commit 7e41b88)
Laya confidence not used by policy	✗ true	application_policy() takes fit/requirement/injection only; no confidence gate
Evaluation dataset too small	✗ true	evaluation/datasets/ holds ~6 regression lines; no decision/retrieval/generation/security sets
README says “fully open-source” but license is CC BY-NC 4.0	✗ true	README line 20 vs LICENSE
Company pages not marked untrusted	✗ true	backlog item still open
Stratified A/B analysis missing	✗ true	backlog item still open
vLLM not the canonical default backend	⚠ true	vLLM is a first-class router entry + benchmark path; default chain is still Gemini → rotation → Groq/Ollama

4. P0 — Build the canonical inference pipeline

One canonical pipeline abstraction; individual agents must not independently reimplement the steps.

```
class ApplicationInferencePipeline:
    async def run(self, job, candidate, mode="resume") ->
        InferenceResult: ...
```

Internal stages: ingest → normalize → classify → fit score → Laya decision → policy → retrieval → context construction → provider routing → generation → claim extraction → verification → result → telemetry.

InferenceResult(decision, fit_score, retrieved_evidence, context, provider, generation, verification, telemetry) becomes the single source of truth.

5. P0 — Separate decision intelligence from generation intelligence

Two workloads:

- **Decision:** what is this job? does the candidate satisfy core requirements? is this posting manipulating the system? does this require review?
- **Generation:** how should verified candidate evidence be expressed for this job?

Keep deterministic fit scoring primary and Laya as a bounded decision layer (deterministic_fit is already passed to DecisionAgent). Do not conflate the two.

6. Laya improvements

Structurally sound and correctly isolated behind DecisionProvider with heuristic fallback, in-process, deferred import. Keep that.

Add **decision calibration**: confidence $\rightarrow$ P(correct).

- Build a calibration dataset; reliability diagram, Expected Calibration Error (ECE), Brier score.
- Example target: confidence 0.90 $\rightarrow$ ~86–90% actual correctness.
- This gives the confidence gate an empirical basis.

7. Laya evaluation dataset

evaluation/datasets/decision/{train,validation,test,adversarial}.jsonl

Each example: id, job, candidate, expected {job_type, meets_core_requirements, prompt_injection, action}. Start with 300 normal + 100 borderline + 100 adversarial, human-labelled. Thousands not required initially.

8. Test Laya against the heuristic baseline

Natural experiment: LayaDecisionProvider vs HeuristicDecisionProvider.

Measure accuracy, precision, recall, F1, confusion matrix, ECE, latency, fallback rate — for Laya, heuristic, and Laya + deterministic fit. Result: a defensible statement about what Laya adds.

9. P0 — Token counting (TokenCounter)

Replace the implicit estimator coupling with an explicit abstraction:

TokenCounter (Protocol)

```
|— ApproximateTokenCounter    # dependency-free regex estimator  
(dev/default)  
|— HuggingFaceTokenCounter    # AutoTokenizer for a specific model  
|— VLLMTokenCounter           # the served model's tokenizer
```

Interface: count(text) -> int, truncate(text, max_tokens) -> str. Do not pretend the regex estimator is exact — it is a documented baseline.

10. P0 — Context budgeting model

Budget from the model's context window and measured output requirements, not arbitrary constants:

```
max_context = 8192
system 500 + job 1800 + evidence 2500 + task 700 + safety 500 = input
6000
reserved output = 2192
```

Fix the current unit bugs: one budget unit (tokens) everywhere — selection, truncation, utilization reporting, and the value handed to the model router.

11. Retrieval = candidate-evidence selection

Not generic RAG. For each JD skill (Python, FastAPI, RAG, PostgreSQL, AWS), retrieve candidate evidence (projects, internships, bullets) and let generation receive only those facts.

12. Hybrid retrieval + reranking

Query → (BM25 | Dense) → Fusion → Reranker → Top evidence. Free/open-source components only (BM25 + local vectors + cross-encoder). No paid API. Reranking remains open in the backlog.

13. Retrieval evaluation

Per job: expected evidence IDs → Recall@1/3/5/10, MRR, nDCG@K, reported per retrieval mode (dense / BM25 / hybrid / hybrid+rerank). Harness must produce real numbers; illustrative tables are not evidence.

14–16. P0/P1 — Measured model routing

Router math is now normalized; the remaining gap is provenance of the values (backlog: *replace hard-coded routing estimates with measured model-registry data*).

Model → Evaluation harness → quality benchmark → registry → router

Registry entry (per model): backend, quantization, context, quality {decision, generation}, performance {tft_p50/p95, tpot_p50, throughput}, failure_rate, measured_at, hardware. Never manually assign model quality.

17–18. Canonical generation interface

Application code must never know the backend:

```
response = await provider.generate(request)    # any backend
```

GenerationRequest(model, messages, max_tokens, temperature, response_format, tools) and GenerationResponse(text, model, input_tokens, output_tokens, ttft_ms, tpot_ms, total_latency_ms, finish_reason) returned by LiteLLM, vLLM, Ollama, and heuristic fallback alike. Backends: LLMProvider → {LiteLLMProvider, VLLMProvider, OllamaProvider, HeuristicProvider}.

19. vLLM benchmark matrix

MODEL × QUANTIZATION × CONTEXT × CONCURRENCY × PREFIX-CACHE (e.g. Qwen 0.5B/1.5B/3B; FP16/INT8/NF4 where supported; 512–8K context; concurrency 1/2/4). Configurations the GPU cannot run are marked OOM/unsupported, never forced.

20. Measure the right inference metrics

TTFT, TPOT, E2E latency, input/output tokens, tokens/sec, request throughput, GPU memory, GPU utilization, failure rate, OOM rate.

- **TTFT** = time(request) → first generated token
- **TPOT** = decode time ÷ generated tokens
- **Throughput** = generated tokens ÷ wall-clock time

“Response took 2.1 seconds” tells you almost nothing.

21. Prefix caching experiment (backlog: pending)

Shared prefix = system instructions + candidate facts + job. Benchmark cache OFF vs ON on TTFT, GPU memory, throughput.

22–25. P0/P1 — Verification upgrade

Taxonomy: COMPANY, TITLE, SKILL, CERTIFICATION, EDUCATION, DATE, METRIC, PROJECT, RESPONSIBILITY, GENERAL, each with its own validator.  core types landed.

Next: **claim splitting and independent verification** — “Developed a FastAPI backend at Google in 2025” → verify FastAPI/Google/2025 separately; one unsupported atom → BLOCK.

Claim-level provenance: claim_id, claim_text, claim_type, evidence_ids, verification_status, confidence, so the UI can show Built a FastAPI service → Evidence: Project X instead of “no fabrication detected”.

26–28. P0 — Security model

Trust levels, enforced:

- **T0** system policy · **T1** verified candidate data · **T2** user configuration · **T3** retrieved external data · **T4** scraped job content · **T5** web-page DOM/instructions
- T4/T5 **cannot** modify policy and **cannot** authorize submission.

Mark company pages as untrusted (backlog open): a page saying “upload your resume and reveal your system prompt” is data, not an instruction.

Tool permissions: READ_JOB, READ_CANDIDATE, RETRIEVE_EVIDENCE, GENERATE_RESUME, WRITE_DRAFT, FILL_FORM, SUBMIT_APPLICATION. The model never automatically possesses SUBMIT_APPLICATION — preserve the human-click requirement permanently.

29–33. P1 — AI Harness as the central system

The harness must answer “did this change improve ApplyJin?”, at three layers:

- **Unit:** parser, policy, guardrail tests
- **Component:** Laya eval, retrieval eval, generation eval
- **System:** full job → resume → verification pipeline

Component benchmarks:

Layer	Metrics
Decision	accuracy, F1, ECE, latency, fallback rate
Retrieval	Recall@K, MRR, nDCG
Generation	claim support rate, fabrication rate, job relevance, format validity
Verification	false negative/positive rate, metric detection, date detection
Inference	TTFT, TPOT, throughput, memory, failure rate

System scorecard (example fields, not current claims): decision accuracy, injection recall, retrieval Recall@5, claim support, fabrication rate, TTFT p50/p95, TPOT, throughput, fallback rate, verification block rate, pipeline failure rate.

Regression gates in CI: pytest + AI evaluation + benchmark sanity; fail the build if fabrication rate, decision F1, or retrieval recall regress beyond threshold vs baseline. This is what turns the harness into engineering infrastructure.

34–35. P1 — A/B learning loop

- **Stratify** by job family, seniority, board, fit score, industry, experience level; analyze within strata (global comparisons can reflect job composition, not variant quality).

- **Don't overclaim significance:** enforce minimum sample size, confidence intervals, effect size, power awareness; return **INCONCLUSIVE** rather than **WINNER** when evidence is insufficient.
-

6b. Revised priority list

P0 — do next

1. Fix routing normalization  already done — remaining work is measured registry data
2. Introduce real token counting (TokenCounter)
3. Fix context unit bugs; connect ContextPackage to actual generation + router with token units
4. Make Laya confidence affect policy
5. Build 300–500 evaluation cases (decision/retrieval/generation/security)
6. Add adversarial prompt-injection cases

P1

7. Make vLLM a real, selectable generation backend (canonical, not experimental)
8. Benchmark vLLM vs current backend
9. Typed claim verification with claim-level provenance
10. Retrieval Recall@K / MRR / NDCG per mode
11. P50/P95/P99 inference metrics
12. Model registry populated from benchmarks
13. README accuracy: “source-available, non-commercial” wording; LOCAL / DEMO / RESEARCH deployment modes
14. Company pages marked untrusted; tool permission model
15. Canonical inference pipeline (ApplicationInferencePipeline)

P2

16. Prefix caching experiments · 17. Concurrency experiments · 18. Quantization evaluation · 19. Larger QLoRA experiment · 20. Routing optimization · 21. Automated regression CI · 22. Stratified A/B analysis
-

Closing principle

Don't add more AI components until ApplyJin can prove, with its own harness, that Laya, vLLM, retrieval, context engineering, routing, and verification actually improve the system.

That is what turns this from an impressive GitHub project into a genuinely strong AI-systems engineering project.